

Name: Irfana M

Reg No: 3122235002049

Class: IT-A

UIT2602- Web Programming

Exercise-9

1.You are developing a registration form for a website. The form requires users to enter:

- A username (at least 5 characters)
- An email (must be a valid email format)
- A password (at least 8 characters, must include a number)
- A confirm password field (must match the password)

Problem Statement: Write JavaScript code to validate the form. If any field is incorrect, display an appropriate error message.

AIM

To develop a **JavaScript-based form validation system** that verifies user input in a registration form by checking:

- Username (minimum 5 characters)
- Email (valid email format)
- Password (minimum 8 characters and must contain at least one Number) & Confirm Password (must match password)

PROCEDURE

Step 1: Create HTML Registration Form Create input fields for:

- Username
- Email
- Password
- Confirm Password
- Submit button

Step 2: Add Error Message Elements

Add `<span>` elements below each input field to display validation error messages.

Step 3: Write JavaScript Validation Function

- Prevent form submission using `event.preventDefault()` ☐ Check each condition using:
 - Length validation
 - Regular expressions for email and password
 - Matching password comparison
- Display appropriate error messages if validation fails

Step 4: Link JavaScript to HTML

Use `<script>` tag or external JS file.

CODE

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Registration Form</title>
```

```
  <style>
```

```
body {      font-
family: Arial, sans-
serif;
```

```
        background-color: #f2f2f2;    display:
flex;
        justify-content: center;    align-
items: center;    height: 100vh;
    }
```

```
.form-container {    background-color:
#ffffff;    padding: 30px;    width:
350px;    border-radius: 8px;    box-
shadow: 0 0 10px rgba(0, 0, 0, 0.1);
    }    h2 {    text-
align: center;    margin-bottom:
20px;    color:
#333;
        }    input {
width: 100%;    padding:
10px;    margin-top: 8px;
border: 1px solid #ccc;
border-radius:
4px;
        }
        input:focus {
            border-color: #4CAF50;
outline: none;
        }
    }
```

```
        button {          width: 100%;
padding: 10px;          margin-top:
15px;          background-color:
#4CAF50;          color: white;
border: none;          borderradius:
4px;          cursor: pointer;
        }
```

```
        button:hover {
background-color: #45a049;
        }
```

```
        .error {
color: red;          fontsize:
13px;
        }
```

```
        .success {
color: green;
textalign: center;
margin-top: 10px;  }
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="form-container">
```

```
<h2>Registration Form</h2>
```

```
<form id="registrationForm">
```

```
        <input type="text" id="username" placeholder="Username">        <span
id="usernameError" class="error"></span>
```

```
<input type="text" id="email" placeholder="Email">
<span id="emailError" class="error"></span>
<input type="password" id="password" placeholder="Password">
<span id="passwordError" class="error"></span>
<input type="password" id="confirmPassword" placeholder="Confirm
Password">
<span id="confirmPasswordError" class="error"></span>
<button type="submit">Register</button>
```

```
<div id="successMessage" class="success"></div>
```

```
</form>
```

```
</div>
```

```
<script>
```

```
document.getElementById("registrationForm").addEventListener("submit",
function(event) {
```

```
    event.preventDefault();
```

```
    let username = document.getElementById("username").value.trim();
```

```
    let email = document.getElementById("email").value.trim();    let
```

```
    password = document.getElementById("password").value;
```

```
    let confirmPassword = document.getElementById("confirmPassword").value;
```

```
    // Clear previous errors
```

```
    document.getElementById("usernameError").innerText = "";
```

```
    document.getElementById("emailError").innerText = "";
```

```

document.getElementById("passwordError").innerText = "";
document.getElementById("confirmPasswordError").innerText = "";
document.getElementById("successMessage").innerText = "";    let isValid =
true;

    // Username validation    if
(username.length < 5) {
        document.getElementById("usernameError").innerText =
"Username must be at least 5 characters long.";    isValid =
false;    // Email validation
        let emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;    if
(!emailPattern.test(email)) {
            document.getElementById("emailError").innerText =
"Please enter a valid email address.";    isValid = false;
        }

        // Password validation (min 8 chars + 1 number) let
passwordPattern = /^(?=.*\d){8,}$/;
        if (!passwordPattern.test(password)) {
            document.getElementById("passwordError").innerText =
"Password must be at least 8 characters and include a number.";
            isValid = false;
        }

        // Confirm password validation    if (password
!== confirmPassword) {
            document.getElementById("confirmPasswordError").innerText =

```

```
        "Passwords do not match.";    isValid
= false;
    }
    // If everything correct
    if (isValid) {
        document.getElementById("successMessage").innerText =
            "Registration Successful!";
        document.getElementById("registrationForm").reset();
    }
});
</script>
</body>
</html>
```

Output



A screenshot of a web registration form titled "Registration Form". The form contains four input fields: a username field with the text "Abinaya", an email field with the text "esakkiabinaya2310703@ssn.edu.in", a password field with seven dots, and a confirm password field with seven dots and a toggle eye icon on the right. Below the input fields is a green "Register" button. At the bottom of the form, the text "Registration Successful!" is displayed in green.

JUSTIFICATION

- Prevents invalid data submission.
- Enhances user experience by providing instant feedback.
- Uses Regular Expressions (Regex) for accurate pattern validation.
- Ensures security by enforcing strong password rules.
- Reduces server-side validation load by catching errors on client side.

CONCLUSION

The JavaScript form validation successfully ensures that:

- Username contains at least 5 characters.
- Email follows valid format.
- Password is strong (minimum 8 characters with a number).
- Confirm password matches the original password.

This improves data accuracy, enhances security, and provides better user interaction before submitting data to the server.

2. You are designing a simple quiz app where:

- The user selects the correct answer from multiple-choice questions.
- After the quiz, the total score is displayed.

Problem Statement: Write JavaScript code to create a dynamic quiz with at least 3 questions.

AIM

To develop a **dynamic quiz application** using **JavaScript** where users can select answers for multiple-choice questions and view their total score after submission.

PROCEDURE Step 1: Create HTML Structure

- Add a container to display questions dynamically.
- Add a Submit button.
- Add a section to display the final score.

Step 2: Create Questions in JavaScript

- Store at least 3 questions in an array of objects.
- Each object contains:
 - Question text
 - Options
 - Correct answer

Step 3: Display Questions Dynamically

- ▣ Use JavaScript DOM manipulation to generate question elements and radio buttons.

Step 4: Calculate Score

- ▣ On clicking Submit:
 - Check selected answers
 - Compare with correct answers
 - Calculate total score

Step 5: Display Final Result

CODE

```
<!DOCTYPE html>

<html>

<head>

  <title>Simple Quiz</title>

  <style>

body {

  font-family: Arial, sans-serif;

background-color: #f2f2f2;

display: flex;      justify-content:

center;      padding-top: 40px;

  }

  .quiz-container {      background:

#ffffff;      width: 450px;      padding:

25px;      border-radius: 8px;      boxshadow:

0 0 10px rgba(0,0,0,0.1);

  }

h2 {

text-align

: center;

margin-bottom

: 20px;

  }

  .question {      margin-bottom:

15px;
```

```

    }    label {        display:
        block;        margin:    5px
        0;        cursor: pointer;
    }
    button {        width: 100%;
padding: 10px;        margin-top:
10px;        background-color:
#4CAF50;        color: white;
border: none;        borderradius:
5px;        cursor: pointer;
    }
    button:hover {
background-color: #45a049;
    }

    #result {    margin-top:
    15px;    fontweight: bold;
    text-align:
    center;
    }
</style>
</head>
<body>
<div class="quiz-container">
    <h2>Simple Quiz</h2>
    <form id="quizForm"></form>

```

```

<button onclick="calculateScore()">Submit</button>

<div id="result"></div>
</div> <script> const
quizData = [
  {
    question: "1. Which language is used for web development?",
options: ["Python", "JavaScript", "C++", "Java"],    correct:
"JavaScript"
  },
  {
    question: "2. What does HTML stand for?",
    options: ["Hyper Text Markup Language", "High Text Machine Language",
"Hyper Tool Multi Language", "None"],    correct:
"Hyper Text Markup Language"
  },
  {
    question: "3. Which tag is used to include JavaScript in HTML?",
    options: ["<js>", "<script>", "<javascript>", "<code>"], correct:
"<script>"
  }
];

const quizForm = document.getElementById("quizForm");

// Display questions dynamically (SAFE method)
quizData.forEach((q, index) => {  const div =
document.createElement("div");
div.classList.add("question");

```

```

    const questionTitle = document.createElement("p");
questionTitle.textContent = q.question;    div.appendChild(questionTitle);

    q.options.forEach(option => {

        const label = document.createElement("label");

        const radio = document.createElement("input");
radio.type = "radio";    radio.name = "question" +
index;    radio.value = option;
label.appendChild(radio);

        const optionText = document.createTextNode(" " + option);
label.appendChild(optionText);    div.appendChild(label);

    });

quizForm.appendChild(div);
});

function calculateScore() {    let score = 0;
quizData.forEach((q, index) => {    const
selected = document.querySelector(
        `input[name="question${index}"]:checked`
    );

    if (selected && selected.value === q.correct) {        score++;

    }

});

document.getElementById("result").innerText =
    "Your Score: " + score + " / " + quizData.length;
}

```

</script>

</body>

</html>

Output

Simple Quiz

1. Which language is used for web development?

- ☐ Python
- ☒ JavaScript
- ☐ C++
- ☐ Java

2. What does HTML stand for?

- ☒ Hyper Text Markup Language
- ☐ High Text Machine Language
- ☐ Hyper Tool Multi Language
- ☐ None

3. Which tag is used to include JavaScript in HTML?

- ☐ <js>
- ☒ <script>
- ☐ <javascript>
- ☐ <code>

Submit

Your Score: 3 / 3

JUSTIFICATION

- Uses dynamic question generation (not hardcoded in HTML).
- Demonstrates DOM manipulation.
- Implements event handling.
- Uses arrays and objects effectively.
- Provides interactive user experience.
- Can be easily extended to add more questions.

CONCLUSION

The quiz application successfully allows users to:

- Select answers from multiple-choice questions.
- Submit their responses.
- View their total score instantly.

This project demonstrates practical usage of JavaScript for dynamic content generation, user interaction handling, and score calculation.

3. You are working on an e-commerce website that offers limited-time deals. The timer should count down from 5 minutes and disable the "Buy Now" button when time runs out.

Problem Statement: Write a jQuery script to implement the countdown timer and disable the button when the timer reaches 00:00.

AIM

To develop a **countdown timer using jQuery** that starts from **5 minutes (05:00)** and automatically disables the **"Buy Now"** button when the time reaches **00:00**.

PROCEDURE Step 1: Create HTML Structure

- Add a timer display section.
- Add a "Buy Now" button.

Step 2: Include jQuery

- Add jQuery CDN link in the <head> section.

Step 3: Initialize Timer

- Set total time = 5 minutes (300 seconds).

Step 4: Use setInterval()

- Decrease time every 1 second.
- Convert seconds to minutes & seconds format (MM:SS).

Step 5: Disable Button ⓘ When

- Timer reaches 00:00: ◦ Stop the timer
- Disable the button
- Change button text to "Deal Expired"

COMPLETE CODE (jQuery Countdown Timer)

```
<!DOCTYPE html>
<html>
<head>
  <title>Limited Time Deal</title>
  <!-- jQuery CDN -->
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
  <style>
body {
  font-family: Arial, sans-serif;      background-
color: #f4f4f4;
  text-align: center;
  padding-top: 100px;
}
```



```

        .deal-container {
background: white;
width: 300px;      margin:
auto;      padding: 25px;
border-radius: 8px;
        box-shadow: 0 0 10px rgba(0,0,0,0.1);
        }
        #timer {      fontsize:
28px;      fontweight:
bold;      color: red;
marginbottom: 20px;

        }
        #buyBtn {      padding: 10px
20px;      font-size: 16px;
background-color: green;
        color:      white;      border:
        none;
border-radius: 5px;
        cursor: pointer;
        }

        #buyBtn:disabled {      background-color:
gray;
        cursor: not-allowed;
        }

```

```

</style>

```

```

</head>

```

```

<body>

```

```

<div class="deal-container">

```

```

    <h2> ⚡ Limited Time Offer</h2>

```

```

    <div id="timer">05:00</div>

```

```

    <button id="buyBtn">Buy Now</button>

```

```

</div>

```

```

<script>

```

```

$(document).ready(function() {

```

```

let totalTime = 300; // 5 minutes in seconds

let countdown = setInterval(function() {

    let minutes = Math.floor(totalTime / 60);
    let seconds = totalTime % 60;

    // Add leading zero    minutes = minutes < 10 ?
    "0" + minutes : minutes;    seconds = seconds < 10 ?
    "0" + seconds : seconds;
    $("#timer").text(minutes + ":" + seconds);    totalTime--;    if
    (totalTime < 0) {        clearInterval(countdown);
    $("#timer").text("00:00");
        $("#buyBtn").prop("disabled", true);
        $("#buyBtn").text("Deal Expired");
    }

}, 1000);
});
</script>
</body> </html>

```

Output



JUSTIFICATION

- Uses **jQuery DOM manipulation** for updating content dynamically.

- Uses setInterval() for real-time countdown.
- Formats time properly in MM:SS format.
- Automatically disables the button after time expires.
- Improves user experience by visually showing deal expiry.

CONCLUSION

The countdown timer successfully:

1. Starts from 5 minutes
2. Updates every second
3. Displays time in MM:SS format
4. Disables the "Buy Now" button at 00:00
5. Prevents users from purchasing after deal expiry

4. You are building a real-time search feature for a blog website. When a user types in the search box, matching words in a list of articles should be highlighted in yellow.

Problem Statement: Write a jQuery script that:

- Filters and displays matching results in real-time.
- Highlights the matching text.

AIM

To develop a **real-time search feature using jQuery** that:

- Filters blog article titles as the user types
- Displays matching results instantly
- Highlights matching text in **yellow**

PROCEDURE Step 1: Create HTML Structure

- Add a search input box
- Add a list of article titles

Step 2: Include jQuery

- Add jQuery CDN in the <head> section

Step 3: Capture User Input

- Use keyup() event to detect typing

Step 4: Filter Articles

- Compare search text with article titles
- Show matching results
- Hide non-matching results

Step 5: Highlight Matching Text

- Use replace() with RegEx
- Wrap matching text inside with yellow background

COMPLETE CODE (Real-Time Search with Highlight)

```
<!DOCTYPE html>
<html>
<head>
  <title>Real-Time Blog Search</title>

  <!-- jQuery CDN -->
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
  <style>
body {
    font-family: Arial, sans-serif;
padding: 40px;
    background-color: #f4f4f4;
  }
```

```

        .container {
width: 400px;
margin: auto;
background: white;
padding: 20px;      border-radius:
8px;
        box-shadow: 0 0 10px rgba(0,0,0,0.1);
        }      input {
width: 100%;
padding: 8px;
        margin-bottom: 15px;
        }
        ul {
list-style: none;
padding: 0;
        }
        li {
padding: 8px;
border-bottom: 1px solid #ddd;
        }
        .highlight {
background-color: yellow;      font-
weight: bold;
        }
</style>
</head>
<body>
<div class="container">
    <h2>Blog Search</h2>
    <input type="text" id="search" placeholder="Search articles...">
    <ul id="articleList">
        <li>JavaScript Basics for Beginners</li>
        <li>Understanding jQuery Effects</li>
        <li>Learn HTML and CSS</li>
        <li>Advanced JavaScript Concepts</li>
        <li>Building Dynamic Websites</li>

```

```

    </ul>
</div>
<script>
$(document).ready(function(){
$("#search").on("keyup", function(){
    let searchText = $(this).val().toLowerCase();

    $("#articleList li").each(function(){

        let originalText = $(this).text();
        let lowerText = originalText.toLowerCase();

        if(lowerText.includes(searchText) && searchText !== "") {

            let regex = new RegExp("(" + searchText + ")", "gi");           let
highlightedText = originalText.replace(regex,
            "<span class='highlight'>$1</span>");

            $(this).html(highlightedText).show();

        } else if(searchText === "") {

            $(this).html(originalText).show();

        } else {

            $(this).hide();
        }

    });

});

});
</script>

```

</body> </html>

Output

Blog Search

JavaScript Basics for Beginners

Understanding jQuery Effects

Learn HTML and CSS

Advanced JavaScript Concepts

Building Dynamic Websites

Blog Search

Advanced JavaScript Concepts

JUSTIFICATION

- Uses **jQuery keyup event** for real-time detection
- Uses `includes()` for filtering
- Uses **Regular Expressions (RegEx)** for highlighting
 - Dynamically updates the DOM

- Improves user experience with instant feedback

CONCLUSION The real-time search feature successfully:

- 1.Filters matching blog articles instantly
- 2.Highlights matching text in yellow
- 3.Hides non-matching results
- 4.Updates dynamically as user types